

Name bhavya rana

Sapid 590034689

Batch 15

Algorithm

1. Start.
2. Input the order n of the square matrix.
3. Validate that $n > 0$.
4. Declare a variable-length integer array $a[n][n]$.
5. Read all matrix elements.
6. Repeatedly display the menu:
 - **1:** Check whether all main diagonal elements are distinct.
 - **2:** Find the sum of main diagonal elements.
 - **3:** Display the matrix using diagonal traversal.
 - **4:** Exit.
7. For option 1:
 - Compare every main diagonal element $a[i][i]$ with subsequent diagonal elements.
 - If any two are equal, report that the elements are not distinct.
 - Otherwise, report that all are distinct.
8. For option 2:
 - Add $a[i][i]$ for $i = 0$ to $n-1$.
 - Display the sum.
9. For option 3:
 - Process diagonals according to $row + column = d$, where d ranges from 0 to $2n-2$.
 - For even-numbered diagonals, traverse upward-right.
 - For odd-numbered diagonals, traverse downward-left.
10. For option 4, terminate the program.
11. For any other choice, display an invalid-choice message.
12. Stop.

Test Case Table

Test Case	Input	Operation	Expected Output
1	$n=3$, matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	Distinct diagonal	$1 \ 5 \ 9 \rightarrow$ all distinct
2	Same matrix	Diagonal sum	15

3	Same matrix	Diagonal traversal	1 2 4 7 5 3 6 8 9
4	n=3, matrix 1 2 3 / 4 1 6 / 7 8 9	Distinct diagonal	1 1 9 → not distinct
5	n=1, matrix 7	All operations	Distinct; sum = 7; traversal = 7
6	Any valid matrix, choice 5	Invalid choice	Invalid choice.
7	n=0 initially	Input validation	Request a positive order

C Program

```
#include <stdio.h>

/* Function prototypes */
void checkDistinct(int n, int a[n][n]);
void diagonalSum(int n, int a[n][n]);
void diagonalTraversal(int n, int a[n][n]);

int main()
{
    int n, i, j, choice;

    /* Read and validate matrix order */
    do
    {
        printf("Enter the order of the square matrix: ");
        scanf("%d", &n);

        if (n <= 0)
            printf("Order must be a positive integer.\n");
    } while (n <= 0);

    /* Variable-length 2D array */
    int a[n][n];

    printf("Enter the matrix elements:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}
```

```

do
{
    printf("\n----- MENU ----- \n");
    printf("1. Check whether main diagonal elements are
distinct\n");
    printf("2. Find sum of main diagonal elements\n");
    printf("3. Display diagonal traversal\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            checkDistinct(n, a);
            break;

        case 2:
            diagonalSum(n, a);
            break;

        case 3:
            diagonalTraversal(n, a);
            break;

        case 4:
            printf("Exiting the program.\n");
            break;

        default:
            printf("Invalid choice. Please enter 1 to
4.\n");
    }

    } while (choice != 4);

    return 0;
}

/* Check whether all main diagonal elements are distinct */
void checkDistinct(int n, int a[n][n])
{
    int i, j;
    int distinct = 1;

```

```

printf("Main diagonal elements: ");

for (i = 0; i < n; i++)
{
    printf("%d ", a[i][i]);
}

for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (a[i][i] == a[j][j])
        {
            distinct = 0;
            break;
        }
    }

    if (!distinct)
        break;
}

if (distinct)
    printf("\nAll the main diagonal elements are
distinct.\n");
else
    printf("\nAll the main diagonal elements are not
distinct.\n");
}

/* Find the sum of main diagonal elements */
void diagonalSum(int n, int a[n][n])
{
    int i;
    int sum = 0;

    for (i = 0; i < n; i++)
    {
        sum += a[i][i];
    }

    printf("Sum of main diagonal elements: %d\n", sum);
}

```

```

/* Display elements in alternating diagonal traversal */
void diagonalTraversal(int n, int a[n][n])
{
    int d, i, j;

    printf("Diagonal traversal: ");

    for (d = 0; d <= 2 * n - 2; d++)
    {
        if (d % 2 == 0)
        {
            /* Upward-right traversal */
            i = (d < n) ? d : n - 1;
            j = d - i;

            while (i >= 0 && j < n)
            {
                printf("%d ", a[i][j]);
                i--;
                j++;
            }
        }
        else
        {
            /* Downward-left traversal */
            j = (d < n) ? d : n - 1;
            i = d - j;

            while (j >= 0 && i < n)
            {
                printf("%d ", a[i][j]);
                i++;
                j--;
            }
        }
    }

    printf("\n");
}

```

Execution

```

Last login: Sun Sep 27 19:55:08 on ttys000
bhavya@Bhavyas-MacBook-Air ~ % nano day35.c
bhavya@Bhavyas-MacBook-Air ~ % clang day35.c -o day35
bhavya@Bhavyas-MacBook-Air ~ % ./day35
Enter the order of the square matrix: 3
Enter the matrix elements:
  1 2 3
  4 5 6
  7 8 9

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 15

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 4
Exiting the program.
bhavya@Bhavyas-MacBook-Air ~ %

```

This traversal matches the **exact order specified in the question.**